Data Handling: Import, Cleaning and Visualisation

Solutions to Exercise/Workshop 3: Data types, text files and data import

Dr. Aurélien Sallin (Lecturer) Ana Armendariz (Tutor)

15 October 2026

Table of contents

Exercise 1: types, and how R converts between them 1

Exercise 2: the structures 3

Exercise 3: a text file is just bytes 5

Exercise 4: importing a CSV in two ways 6

Exercise 5: column types 8

Exercise 6: a file that is not comma separated 9

Exercise 7: select what you need and write it back 10

Additional exercises for training 12

Exercise 8: the same data in XML . . . 12

Exercise 9: exploring HTML in the browser . . . 14

Exercise 10: scraping a table from a web page . . . 14

Exercise 1: types, and how R converts between them

1. For the first four, `typeof()` and `class()` say nearly the same thing:

```
c(typeof(TRUE), class(TRUE))
```

```
[1] "logical" "logical"
```

```
c(typeof(3L), class(3L))
```

```
[1] "integer" "integer"
```

```
c(typeof(3.5), class(3.5))
```

```
[1] "double"  "numeric"
```

```r
c(typeof("a"), class("a"))
```

```
[1] "character" "character"
```

The only difference is the name: a `double` has class `numeric`. The factor is the interesting case:

```r
c(typeof(factor("a")), class(factor("a")))
```

```
[1] "integer" "factor"
```

It is stored as an integer and treated as a factor. `typeof()` looks at the storage, `class()` at the behaviour. Question 4 is what happens when you forget the difference.

2. A comparison returns logical values, and R turns them into 1 and 0 when it needs numbers:

```r
my_vector <- c(1, 0, 3, -1)
my_vector > 0
```

```
[1]  TRUE FALSE  TRUE FALSE
```

```r
as.numeric(my_vector > 0)
```

```
[1] 1 0 1 0
```

```r
sum(my_vector > 0)
```

```
[1] 2
```

```r
mean(my_vector > 0)
```

```
[1] 0.5
```

`TRUE` becomes 1 and `FALSE` becomes 0. So on a logical vector `sum()` counts the `TRUE` values and `mean()` gives their share. Two of the four values are positive, which gives 2 and 0.5. `sum(x > 5)` and `mean(x > 5)` are the short way to ask how many observations satisfy a condition, and what proportion of them do.

3. Implicit coercion is silent. Explicit coercion warns you when it fails:

```r
c(1, "2", TRUE)
```

```
[1] "1"    "2"    "TRUE"
```

Everything became text, because `character` is the most general type. Note that `TRUE` became `"TRUE"`, not `"1"`.

```r
as.numeric(c("1", "2", "x"))
```

```
Warning: NAs introduced by coercion
```

```
[1]  1  2 NA
```

Here you asked for the conversion, so R converts what it can and reports the rest: `"x"` is not a number, so it becomes `NA`, with the warning *NAs introduced by coercion*. The warning is useful, because it tells you a value was lost. The first case gives no warning at all.

4. The factor:

```
grades <- factor(c("4.5", "5", "6"))
as.numeric(grades)
```

```
[1] 1 2 3
```

A factor stores integer codes plus a table of levels, not the labels. `as.numeric()` gives you the codes, here 1, 2, 3, because the levels are sorted. To get the values you have to go through the labels:

```
levels(grades)
```

```
[1] "4.5" "5"   "6"
```

```
as.numeric(as.character(grades))
```

```
[1] 4.5 5.0 6.0
```

There is no error, only wrong numbers of the right shape. Whenever you convert a factor to a number, check whether you want the code or the label.

Exercise 2: the structures

1. A vector knows its length and its type, and square brackets index it by position:

```
ages <- c(18, 21, 20, 22, 19)
length(ages)
```

```
[1] 5
```

```
typeof(ages)
```

```
[1] "double"
```

```
ages[3]
```

```
[1] 20
```

`18` is a `double`, not an integer. For an integer you need `18L`.

2. R fills the grid column by column:

```r
m <- matrix(1:6, nrow = 2)
m
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```r
dim(m)
```

```
[1] 2 3
```

```r
attributes(m)
```

```
$dim
[1] 2 3
```

The first column is 1, 2, the second 3, 4, the third 5, 6. The only thing that makes `m` a matrix is the attribute `dim = c(2, 3)`. Underneath it is still the vector `1:6`:

```r
c(m)
```

```
[1] 1 2 3 4 5 6
```

A matrix is a vector that knows its shape, which is why it holds one type only.

3. The vector coerces, the list does not:

```r
persons <- c("Anna", "Luca", "Sara")
c(persons, ages)
```

```
[1] "Anna" "Luca" "Sara" "18"   "21"   "20"   "22"   "19"
```

The ages became text, `"18"` and `"21"`, because a vector holds one type and text is the more general one. `mean()` would now fail.

```r
student_info <- list(names = persons, ages = ages)
student_info$ages
```

```
[1] 18 21 20 22 19
```

```r
str(student_info)
```

```
List of 2
 $ names: chr [1:3] "Anna" "Luca" "Sara"
 $ ages : num [1:5] 18 21 20 22 19
```

A list keeps every element with its own type and its own length. In exchange, it is not a table.

4. The data frame is the structure you will work with for the rest of the course:

```r
students <- data.frame(
  name = c("Anna", "Luca", "Sara"),
  age = c(18, 21, 20),
  passed = c(TRUE, FALSE, TRUE)
)
students
```

```
  name age passed
1 Anna  18   TRUE
2 Luca  21  FALSE
3 Sara  20   TRUE
```

```r
str(students)
```

```
'data.frame':   3 obs. of  3 variables:
 $ name  : chr  "Anna" "Luca" "Sara"
 $ age   : num  18 21 20
 $ passed: logi  TRUE FALSE TRUE
```

```r
sapply(students, class)
```

```
       name         age      passed
"character"   "numeric"   "logical"
```

```r
typeof(students)
```

```
[1] "list"
```

`typeof()` says `list`. A data frame is a list whose elements are the columns, with one extra rule: all elements have the same length. That rule is what makes it a table, and it is why each column has its own type while the rows line up across columns. `class()` says `data.frame`, which is what gives it the printing, the `[row, column]` indexing and `nrow()`.

Exercise 3: a text file is just bytes

1. The "ñ" does not survive:

```r
hasta <- readLines("data/hastamanana.txt", warn = FALSE)
hasta
```

```
[1] "Hasta Ma\xf1ana!"
```

R prints `\xf1` because that byte is not a valid character in UTF-8, the encoding it assumed. On some consoles you see `Hasta Ma?ana!` instead.

2. The bytes:

```r
readBin("data/hastamanana.txt", what = "raw", n = 20)
```

```
 [1] 48 61 73 74 61 20 4d 61 f1 61 6e 61 21 0a
```

`48 61 73 74 61` is `Hasta`, `20` is a space, `4d 61` is `Ma`, then `f1`, then `61 6e 61 21` for `ana!`. Every byte except `f1` is below `7f`, which is plain ASCII. The last byte, `0a`, ends the line.

3. With the right encoding the line reads correctly:

```
readLines("data/hastamanana.txt", encoding = "latin1", warn = FALSE)
```

```
[1] "Hasta Mañana!"
```

You can also convert with `iconv()`, which is what you need when the text is already in R:

```
iconv(hasta, from = "ISO-8859-1", to = "UTF-8")
```

```
[1] "Hasta Mañana!"
```

`f1` is "ñ" in ISO-8859-1. UTF-8 writes the same character as `c3 b1`, so a UTF-8 reader meeting a single `f1` has nothing valid to decode.

4. An error stops you and tells you where to look. A wrong encoding gives you a value that looks like data, so the mistake turns up much later, usually in a table with broken names in it.

Exercise 4: importing a CSV in two ways

1. Base R has `read.csv()`, the `readr` package has `read_csv()`:

```
aq_base <- read.csv("data/Air_Quality.csv")

library(readr)
```

```
Warning: package 'readr' was built under R version 4.5.3
```

```
aq <- read_csv("data/Air_Quality.csv", show_col_types = FALSE)
```

2. Just over 16,000 rows and 12 columns:

```
dim(aq)
```

```
[1] 16122    12
```

```
head(aq[, c("Name", "Geo Place Name", "Time Period", "Data Value")])
```

```
# A tibble: 6 x 4
  Name                 `Geo Place Name`                     `Time Period`  `Data Value`
  <chr>                <chr>                                <chr>                 <dbl>
1 Ozone (O3)           Coney Island (CD13)                  Summer 2013            34.6
2 Ozone (O3)           Coney Island (CD13)                  Summer 2014            33.2
3 Ozone (O3)           Bronx                                Summer 2013            31.2
4 Ozone (O3)           Bronx                                Summer 2014            31.2
5 Sulfur Dioxide (SO2) Morris Park and Bronxdale (CD11)     Winter 2008-09          5.89
6 Sulfur Dioxide (SO2) Williamsbridge and Baychester (CD12) Winter 2008-09          5.75
```

```
library(dplyr)
```

```
Warning: package 'dplyr' was built under R version 4.5.3
```

```
glimpse(aq)
```

```
Rows: 16,122
Columns: 12
$ `Unique ID`      <dbl> 216498, 216499, 219969, 219970, 164876, 164877, 219971, 219972, 164878, 1~
$ `Indicator ID`   <dbl> 386, 386, 386, 386, 383, 383, 386, 386, 383, 383, 383, 383, 383, 383, 383~
$ Name             <chr> "Ozone (O3)", "Ozone (O3)", "Ozone (O3)", "Ozone (O3)", "Sulfur Dioxide (~
$ Measure          <chr> "Mean", "Mean", "Mean", "Mean", "Mean", "Mean", "Mean", "Mean", "Mean", "~
$ `Measure Info`   <chr> "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "pp~
$ `Geo Type Name`  <chr> "CD", "CD", "Borough", "Borough", "CD", "CD", "Borough", "Borough", "CD",~
$ `Geo Join ID`    <dbl> 313, 313, 1, 1, 211, 212, 2, 2, 301, 302, 303, 304, 305, 306, 307, 308, 2~
$ `Geo Place Name` <chr> "Coney Island (CD13)", "Coney Island (CD13)", "Bronx", "Bronx", "Morris P~
$ `Time Period`    <chr> "Summer 2013", "Summer 2014", "Summer 2013", "Summer 2014", "Winter 2008-~
$ Start_Date       <chr> "06/01/2013", "06/01/2014", "06/01/2013", "06/01/2014", "12/01/2008", "12~
$ `Data Value`     <dbl> 34.64, 33.22, 31.25, 31.15, 5.89, 5.75, 26.27, 33.83, 4.33, 4.41, 4.73, 4~
$ Message          <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
```

One row is one measurement: one indicator, in one geographic area, over one time period, with one value. The same pollutant appears in many rows.

3. The names are not the same:

```
names(aq_base)
```

```
 [1] "Unique.ID"      "Indicator.ID"   "Name"           "Measure"        "Measure.Info"
 [6] "Geo.Type.Name"  "Geo.Join.ID"    "Geo.Place.Name" "Time.Period"    "Start_Date"
[11] "Data.Value"     "Message"
```

```
names(aq)
```

```
 [1] "Unique ID"      "Indicator ID"   "Name"           "Measure"        "Measure Info"
 [6] "Geo Type Name"  "Geo Join ID"    "Geo Place Name" "Time Period"    "Start_Date"
[11] "Data Value"     "Message"
```

`read.csv()` repairs the names and turns spaces into dots, so `Unique ID` becomes `Unique.ID` and you write `aq_base$Unique.ID`. `read_csv()` keeps the original names, so a name with a space needs backticks:

```
head(aq$`Unique ID`, 3)
```

```
[1] 216498 216499 219969
```

Neither is wrong, but you need to know which one you are using.

4. The objects differ as well:

```r
class(aq_base)
```

```
[1] "data.frame"
```

```r
class(aq)
```

```
[1] "spec_tbl_df" "tbl_df"      "tbl"         "data.frame"
```

`read.csv()` returns a `data.frame`, `read_csv()` returns a `tibble`. A tibble prints only the first ten rows and shows the type of every column under its name. It is still a data frame, which you can see from `data.frame` in its class.

5. For a file you do not know, the message helps: it says how every column was interpreted, so a column of numbers that was read as text shows up immediately instead of three steps later.

Exercise 5: column types

1. The full guess:

```r
spec(aq)
```

```
cols(
  `Unique ID` = col_double(),
  `Indicator ID` = col_double(),
  Name = col_character(),
  Measure = col_character(),
  `Measure Info` = col_character(),
  `Geo Type Name` = col_character(),
  `Geo Join ID` = col_double(),
  `Geo Place Name` = col_character(),
  `Time Period` = col_character(),
  Start_Date = col_character(),
  `Data Value` = col_double(),
  Message = col_logical()
)
```

Four columns were guessed as `col_double()`, the two IDs, the join ID and `Data Value`, and the rest as `col_character()`.

2. `Start_Date` looks like a date to us but not to the parser:

```r
head(aq$Start_Date, 3)
```

```
[1] "06/01/2013" "06/01/2014" "06/01/2013"
```

`readr` recognises only the ISO format, `YYYY-MM-DD`. `06/01/2013` is ambiguous: 1 June 2013 in the American convention, 6 January 2013 in the European one. Instead of guessing, `readr` leaves it as text. With `2013-06-01` in the file, the column would have been parsed as a date without help.

3. Stating the format solves it:

```
aq_dates <- read_csv(
  "data/Air_Quality.csv",
  col_types = cols(Start_Date = col_date(format = "%m/%d/%Y"))
)

class(aq_dates$Start_Date)
```

```
[1] "Date"
```

```
range(aq_dates$Start_Date)
```

```
[1] "2005-01-01" "2020-06-01"
```

The data runs from January 2005 to June 2020. On the text version the answer is different:

```
range(aq$Start_Date)
```

```
[1] "01/01/2005" "12/31/2015"
```

That is an alphabetical sort. The smallest string is `"01/01/2005"` and the largest is `"12/31/2015"`, which is neither the first nor the last date in the data, because a string starting with `12/` always sorts after one starting with `06/`, whatever the year. The answer looks plausible and is wrong.

4. Every later step inherits the type. A date stored as text cannot be sorted, subtracted or plotted on a time axis, and a number stored as text breaks every calculation. Fixing the type at import also writes down, in the script, what you expect the file to contain. The day the provider changes the format, the import fails instead of producing `NA`.

Exercise 6: a file that is not comma separated

1. The file starts with nine lines of metadata: variable, data set, file name, file path, bad flag, subset, longitude, latitude, and one line repeating the coordinates. The first observation, `JAN-1880`, is on line 10.
2. Reading it:

```
ta <- read_tsv("data/temp_anomaly.txt",
               skip = 9,
               col_names = c("month", "temp_anomaly"),
               show_col_types = FALSE)
head(ta)
```

```
# A tibble: 6 x 2
  month    temp_anomaly
  <chr>           <dbl>
1 JAN-1880         0.33
2 FEB-1880         0.3
3 MAR-1880         0.47
4 APR-1880         0.15
5 MAY-1880        -0.36
6 JUN-1880        -0.43
```

3. `skip = 9` drops the first nine lines before parsing starts, so the metadata stays out of the table. `col_names` is needed because the first line after the metadata is already data: there is no header row for `readr` to take the names from. Without it, `JAN-1880` and `0.33` would have become the column names and the series would be one observation short.
4. The separator is a tab, not a comma:

```
ta_wrong <- read_csv("data/temp_anomaly.txt", skip = 9,
                     col_names = "everything", show_col_types = FALSE)
head(ta_wrong, 3)
```

```
# A tibble: 3 x 1
  everything
  <chr>
1 "JAN-1880\t0.33"
2 "FEB-1880\t0.3"
3 "MAR-1880\t0.47"
```

`read_csv()` finds no comma, so it reads each line as one single field: one column, with the tab character `\t` still inside the values. Nothing is separated, and the anomaly is text instead of a number. `read_tsv()` is `read_csv()` with the tab as delimiter, and `read_delim()` lets you give any delimiter, which is what the `:` separated file in the Lecture 4 homework needs.

5. The series and its mean:

```
nrow(ta)
```

```
[1] 1713
```

```
mean(ta$temp_anomaly)
```

```
[1] 0.09798015
```

```
summary(ta$temp_anomaly)
```

```
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
-1.78000 -0.28000  0.07000  0.09798  0.48000  1.79000
```

1713 monthly observations from January 1880 on, with a mean anomaly of about 0.1 K, a minimum of -1.78 and a maximum of 1.79. The largest values are the recent ones.

Exercise 7: select what you need and write it back

1. Nineteen indicators, from ozone to traffic density:

```
length(unique(aq$Name))
```

```
[1] 19
```

```
unique(aq$Name)
```

```
 [1] "Ozone (O3)"
 [2] "Sulfur Dioxide (SO2)"
 [3] "PM2.5-Attributable Deaths"
 [4] "Boiler Emissions- Total SO2 Emissions"
 [5] "Boiler Emissions- Total PM2.5 Emissions"
 [6] "Boiler Emissions- Total NOx Emissions"
 [7] "Air Toxics Concentrations- Average Benzene Concentrations"
 [8] "Air Toxics Concentrations- Average Formaldehyde Concentrations"
 [9] "PM2.5-Attributable Asthma Emergency Department Visits"
[10] "PM2.5-Attributable Respiratory Hospitalizations (Adults 20 Yrs and Older)"
[11] "PM2.5-Attributable Cardiovascular Hospitalizations (Adults 40 Yrs and Older)"
[12] "Traffic Density- Annual Vehicle Miles Traveled"
[13] "O3-Attributable Cardiac and Respiratory Deaths"
[14] "O3-Attributable Asthma Emergency Department Visits"
[15] "O3-Attributable Asthma Hospitalizations"
[16] "Traffic Density- Annual Vehicle Miles Traveled for Cars"
[17] "Traffic Density- Annual Vehicle Miles Traveled for Trucks"
[18] "Nitrogen Dioxide (NO2)"
[19] "Fine Particulate Matter (PM2.5)"
```

2. Three conditions, with `&` between each pair:

```
pm <- subset(aq,
             Name == "Fine Particulate Matter (PM2.5)" &
             `Time Period` == "Winter 2008-09" &
             `Geo Place Name` == "Lower Manhattan")

dim(pm)
```

```
[1]  1 12
```

```
pm$`Data Value`
```

```
[1] 15.79
```

One row is left: in winter 2008-09, fine particulate matter in Lower Manhattan was measured at 15.79 mcg/m³. Use `&` and not `&&`: `&&` compares only the first element and returns a single `TRUE` or `FALSE`, which keeps either everything or nothing.

3. Writing it out:

```
write_csv(pm, "data/airquality_manhattan.csv")
```

The file has a header line with the twelve column names and one line of values. `write_csv()` writes UTF-8 and, unlike `write.csv()`, adds no column of row numbers. If you want only the number instead of a table, use `writeLines(as.character(pm$"Data Value"), "data/airquality_manhattan.txt")`.

4. Dropping the two conditions on the time period and keeping the place and the indicator gives the whole series for that neighbourhood:

```
pm_all <- subset(aq,
                 Name == "Fine Particulate Matter (PM2.5)" &
                 `Geo Place Name` == "Lower Manhattan")

dim(pm_all)
```

```
[1] 36 12
```

With the dates read as dates, the series sorts properly:

```
pm_sorted <- pm_all[order(as.Date(pm_all$Start_Date, format = "%m/%d/%Y")), ]
head(pm_sorted[, c("Time Period", "Start_Date", "Data Value")], 4)
```

```
# A tibble: 4 x 3
  `Time Period`       Start_Date `Data Value`
  <chr>               <chr>             <dbl>
1 Winter 2008-09      12/01/2008         15.8
2 Annual Average 2009 12/01/2008         13.1
3 Summer 2009         06/01/2009         13.2
4 Winter 2009-10      12/01/2009         12.4
```

```
write_csv(pm_sorted, "data/airquality_manhattan_all.csv")
```

On the text version, sorting puts December before February, for the same reason `range()` misbehaved in Exercise 5.

Additional exercises for training

Exercise 8: the same data in XML

1. What comes back is not a data frame:

```
library(xml2)
```

```
Warning: package 'xml2' was built under R version 4.5.3
```

```
doc <- read_xml("data/Air_Quality.xml")
class(doc)
```

```
[1] "xml_document" "xml_node"
```

```
xml_name(doc)
```

```
[1] "row"
```

It is an object that represents the XML tree. It has everything we need, but we cannot use it as a table. The root element is called `row`.

2. Each measurement is a `row` element inside that root, which is why the XPath expression has `row` twice:

```
rows <- xml_find_all(doc, "//row/row")
length(rows)
```

```
[1] 16122
```

16,122 measurements, the same number as the CSV has rows. The two files hold the same data in two formats.

3. The children of one measurement are the variables, and their text is the values:

```
xml_name(xml_children(rows[[1]]))
```

```
 [1] "unique_id"      "indicator_id"   "name"           "measure"        "measure_info"
 [6] "geo_type_name"  "geo_join_id"    "geo_place_name" "time_period"    "start_date"
[11] "data_value"
```

```
xml_text(xml_children(rows[[1]]))
```

```
 [1] "216498"              "386"                 "Ozone (O3)"          "Mean"
 [5] "ppb"                 "CD"                  "313"                 "Coney Island (CD13)"
 [9] "Summer 2013"         "2013-06-01T00:00:00" "34.64"
```

4. One variable for all measurements, with one XPath expression:

```
data_value <- xml_text(xml_find_all(doc, "//row/row/data_value"))
length(data_value)
```

```
[1] 16122
```

```
head(data_value, 5)
```

```
[1] "34.64" "33.22" "31.25" "31.15" "5.89"
```

```
typeof(data_value)
```

```
[1] "character"
```

Everything that comes out of an XML document is text, because XML has no column types. So you have to convert before computing, as in Exercise 1:

```
mean(as.numeric(data_value))
```

```
[1] 19.12916
```

5. The dates and the file sizes:

```
head(xml_text(xml_find_all(doc, "//row/row/start_date")), 2)
```

```
[1] "2013-06-01T00:00:00" "2014-06-01T00:00:00"
```

```
round(file.size("data/Air_Quality.xml") / 1024^2, 1)   # MB
```

```
[1] 8.7
```

```
round(file.size("data/Air_Quality.csv") / 1024^2, 1)   # MB
```

```
[1] 1.9
```

In the XML the dates are ISO timestamps, `2013-06-01T00:00:00`, which `readr` parses as dates without a format string. The CSV has the ambiguous `06/01/2013`.

XML stores structure: elements can be nested, repeated and labelled, every value carries its variable name, and a document can describe its own rules. A CSV can only hold a flat table. You pay for it in size and speed, 8.7 MB against 1.9 MB for the same data, because every value is wrapped in its tags, and you need XPath to get anything out. For a flat table, a CSV is easier.

Exercise 9: exploring HTML in the browser

1. to 3. Every page arrives in your browser as HTML text. In the source you can recognise the structure: `<article>` for a story, `<h3>` for its headline, `<a href="...">` for the link, `<img src="...">` for the image, and `class` attributes that the stylesheet uses to decide how it looks. The headline on the screen is the text between an opening and a closing tag.
2. The page changes immediately, and only on your own screen. The browser keeps a copy of the document in memory, the DOM, and renders that copy. Your edit changes the copy, not the server. Reload the page and the real headline is back. Nobody else saw anything.
3. The image works the same way: paste the address of an image that is online into the `src` attribute of the `<img>` tag, and the browser fetches that one instead.
4. What a website shows is a rendering of data that was sent to you as text, and the rendering can be changed at will. Two consequences. The data behind a page is accessible, which is what makes Exercise 10 possible. And a screenshot of a website is not evidence of anything.

Exercise 10: scraping a table from a web page

The code needs an internet connection, so it is shown here rather than executed.

```
library(rvest)
library(dplyr)
library(stringr)
library(readr)

SOURCE_PATH <- "https://en.wikipedia.org/wiki/Divided_government_in_the_United_States"
OUTPUT_PATH <- "data/divided_gov.csv"

# 1. fetch the whole HTML document
doc <- read_html(SOURCE_PATH)

# 2. extract the tables and pick the right one
tables <- html_table(doc, fill = TRUE)
length(tables)
tab <- tables[[2]]        # the one with Year, President, Senate, House
```

```r
# 3. check the last row and drop it if it is a footnote
tail(tab, 2)
tab <- tab[-nrow(tab), ]

# 4. one row per year instead of one row per Congress
all_years <- list()

for (i in seq_len(nrow(tab))) {
  row <- tab[i, ]
  pieces <- unlist(strsplit(row$Year, split = "[\\-\\-]", perl = TRUE))
  begin <- as.numeric(pieces[1])
  end <- as.numeric(pieces[2])

  all_years[[i]] <- data.frame(year = begin:(end - 1),
                               president = row$President,
                               senate = row$Senate,
                               house = row$House)
}

divided_gov <- bind_rows(all_years)

# 5. remove Wikipedia's footnote markers
divided_gov$senate <- str_remove_all(divided_gov$senate, "\\[[^\\]]*\\]")
divided_gov$house <- str_remove_all(divided_gov$house, "\\[[^\\]]*\\]")

# 6. inspect and write to disk
head(divided_gov)
write_csv(divided_gov, OUTPUT_PATH)
```

The result is also in the `data` folder:

```r
divided_gov <- readr::read_csv("data/divided_gov.csv", show_col_types = FALSE)
dim(divided_gov)
```

```
[1] 164   4
```

```r
head(divided_gov, 4)
```

```
# A tibble: 4 x 4
   year president senate house
  <dbl> <chr>     <chr>  <chr>
1  1861 Lincoln   R      R
2  1862 Lincoln   R      R
3  1863 Lincoln   R      R
4  1864 Lincoln   R      R
```

Three remarks on the method.

`html_table()` returns a list, because a page usually has more than one table, so you have to look at the candidates and choose. `fill = TRUE` pads rows that have fewer cells than the header, which is common in hand-written Wikipedia tables.

The split pattern `"[\\-\\-]"` matches two characters: the ASCII hyphen `-` and the typographic en dash `–`. Wikipedia uses both, and a pattern with only the hyphen fails on part of the rows without saying so. It is the same kind of problem as the encoding in Exercise 3.

`begin:(end - 1)` and not `begin:end`: a Congress from 1861 to 1863 covers the years 1861 and 1862, and 1863 belongs to the next one. Off-by-one errors are the usual mistake when reshaping periods into years, and the way to catch them is to count a couple of rows by hand.